

# Lectura 1: Conceptos Fundamentales en programación.

## Introducción

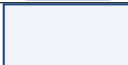
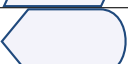
La programación es un elemento fundamental en el mundo tecnológico actual que consiste en dar instrucciones a un computador. Hoy en día, la mayoría de las actividades cotidianas involucran algún tipo de interacción con sistemas computacionales, lo que resalta la importancia de comprender los conceptos básicos de este campo.

Uno de los elementos clave en programación son las variables, las cuales permiten almacenar y manipular datos. Éstas se definen mediante un identificador único y se les asigna un valor que puede ser modificado a lo largo del programa. Las variables pueden ser de diferentes tipos: numéricos enteros o de *punto flotante*, cadenas de caracteres y lógicos entre otros. Además, las variables pueden ser sometidas a diversas operaciones tanto matemáticas como lógicas que permiten realizar cálculos y tomar decisiones a lo largo de la secuencia de instrucciones denominada código. La entrada y salida de datos también son conceptos esenciales en la *programación* ya que son la base de la comunicación entre la máquina y el usuario. Los *algoritmos* requieren de información, proporcionada por el usuario o por otras fuentes, que es procesada y devuelta en forma de resultados. Estos conceptos, junto con la estructura básica de los programas, conforman los cimientos de la programación que todo desarrollador debe dominar.

## Diagrama de flujo

---

Son herramientas gráficas que representan la secuencia de pasos de un algoritmo. Estos diagramas utilizan una simbología estándar que incluye: óvalos para el inicio y fin de los programas, rectángulos para las operaciones, rombos para las decisiones, paralelogramos para las entradas/salidas de datos y flechas para indicar el flujo de las instrucciones.

| Símbolo                                                                             | Nombre         | Función                                                                       |
|-------------------------------------------------------------------------------------|----------------|-------------------------------------------------------------------------------|
|  | Inicio/Fin     | Demarca el inicio o fin de un programa o procedimiento.                       |
|  | Proceso        | Indica las operaciones entre variables e instrucciones para la máquina.       |
|  | Decisión       | Dependiendo el valor de una variable el flujo del programa se puede bifurcar. |
|  | Entrada/salida | Lectura y escritura de datos.                                                 |
|  | Pantalla       | Presentación en pantalla de resultados.                                       |
|  | Flecha         | Dirección de la secuencia de instrucciones.                                   |

Los diagramas de flujo permiten visualizar la estructura lógica de un programa, lo que facilita su diseño, implementación y depuración.

### Ejercicio 1

En conjunto con el docente, elaborar en el tablero el diagrama de flujo para el algoritmo que resuelve el problema de ir del salón de clase a la Biblioteca Departamental en MIO.

## Las variables y sus tipos

---

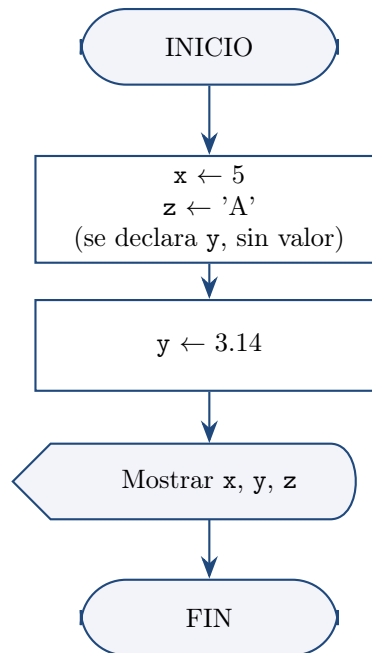
Las variables se identifican por un nombre o una dirección de memoria en donde se almacena su valor. La asignación de valores a las variables es un proceso fundamental en la programación, ya que permite manipular datos y realizar cálculos.

Existen diferentes tipos de variables según el tipo de datos que pueden almacenar, como enteros, punto flotante, lógicos, caracteres y cadenas de texto. Estas se definen generalmente declarando su tipo y su nombre; por ejemplo: `integer :: numero = 5` declara una variable entera llamada `numero` con valor 5 en el lenguaje **Fortran**. El siguiente ejemplo muestra la declaración y asignación de varios tipos de variables; es un código de computadora, es decir, una secuencia de instrucciones.

```
1 program variables
2   implicit none
3   integer :: x = 5           ! Variable entera
4   double precision :: y     ! Variable de punto flotante
5   character :: z = 'A'      ! Variable de caracter
6
7   y = 3.14d0
8   print *, "x = ", x
9   print *, "y = ", y
10  print *, "z = ", z
11
12 end program variables
```

La línea 1 inicia el programa con la palabra reservada `program`, seguida de un nombre elegido por quien programa (aquí, `variables`). La línea 2, `implicit none`, obliga a declarar explícitamente el tipo de cada variable antes de usarla – es una buena práctica que evita errores comunes de digitación. En las líneas 3 a 5 se aparta un lugar en la memoria para tres variables: `x` (entera, con valor inicial 5), `y` (de punto flotante, doble precisión) y `z` (de un solo carácter, con valor inicial `'A'`). En la línea 7 se asigna el valor 3.14 a `y` (el sufijo `d0` indica que la constante es de doble precisión). Las líneas 8, 9 y 10 emplean la instrucción `print` para mostrar en pantalla el valor de cada variable. El programa finaliza con `end program variables`; a diferencia de C++, Fortran no requiere una instrucción de retorno explícita.

El diagrama de flujo correspondiente es el siguiente:



| Variable | x | y    | z   |
|----------|---|------|-----|
| Valor    | 5 | 3.14 | 'A' |

## Ejercicio 2

En la página OnlineGDB, puede copiar el código anterior y ejecutarlo empleando el botón *Run*.

## Entrada y salida de datos

La interacción entre el usuario y el programa es esencial para la mayoría de las aplicaciones. La entrada de datos se refiere a la información que el programa recibe, ya sea del usuario o de otras fuentes. Esto se logra a través de funciones o sentencias específicas proporcionadas por el lenguaje de programación, diseñadas para permitir al usuario introducir datos en el programa al momento de ser ejecutado. Por otro lado, la salida de datos se refiere a la información que el programa entrega al usuario mediante un dispositivo como la pantalla, la impresora o un archivo. Esto también se realiza mediante funciones o sentencias específicas que permiten mostrar los resultados del procesamiento de datos en dichos *dispositivos de salida*. La entrada y salida de datos son conceptos fundamentales en la programación que facilitan la creación de programas interactivos y útiles. En general, la entrada estándar es por teclado pero puede hacerse por disco, USB, Internet, entre otros. La salida estándar se realiza por pantalla pero también puede ser en impresora, archivo, etc.

El siguiente código nos muestra un ejemplo.

```

1 program entrada_salida
2     implicit none
3     integer :: edad    ! Definición de la variable edad
4
5     print *, "Ingrese su edad: "
6     read *, edad       ! Lectura de datos
  
```

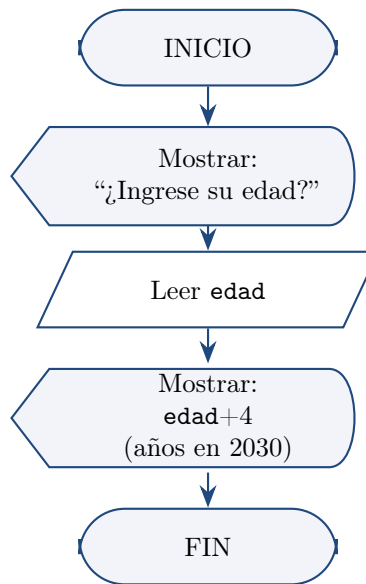
```

7
8   print *, "En el 2030, usted tendra ", edad + 4, " años de edad."
9   ! Impresion de datos
10
11 end program entrada_salida

```

En Fortran, `print *`, ... desempeña el mismo papel que `cout` en C++: envía datos a la salida estándar (la pantalla). De forma análoga, `read *`, `edad` desempeña el papel de `cin`: suspende el programa hasta que el usuario ingresa un valor por teclado y lo guarda en la variable indicada.

El diagrama de flujo correspondiente es el siguiente:



### Ejercicio 3

En la página OnlineGDB, puede copiar el código anterior y ejecutarlo empleando el botón *Run*.

## Operaciones con variables

Las variables pueden ser sometidas a diversas *operaciones matemáticas y lógicas*. Las operaciones matemáticas básicas son la suma, la resta, la multiplicación y la división, que permiten realizar cálculos numéricos. Adicionalmente, existen operaciones lógicas como AND, OR, NOT y XOR, que se utilizan para realizar comparaciones y tomar decisiones en el código. La combinación de variables y operaciones permite construir algoritmos que resuelven problemas de manera eficiente. Considera el siguiente ejemplo.

```

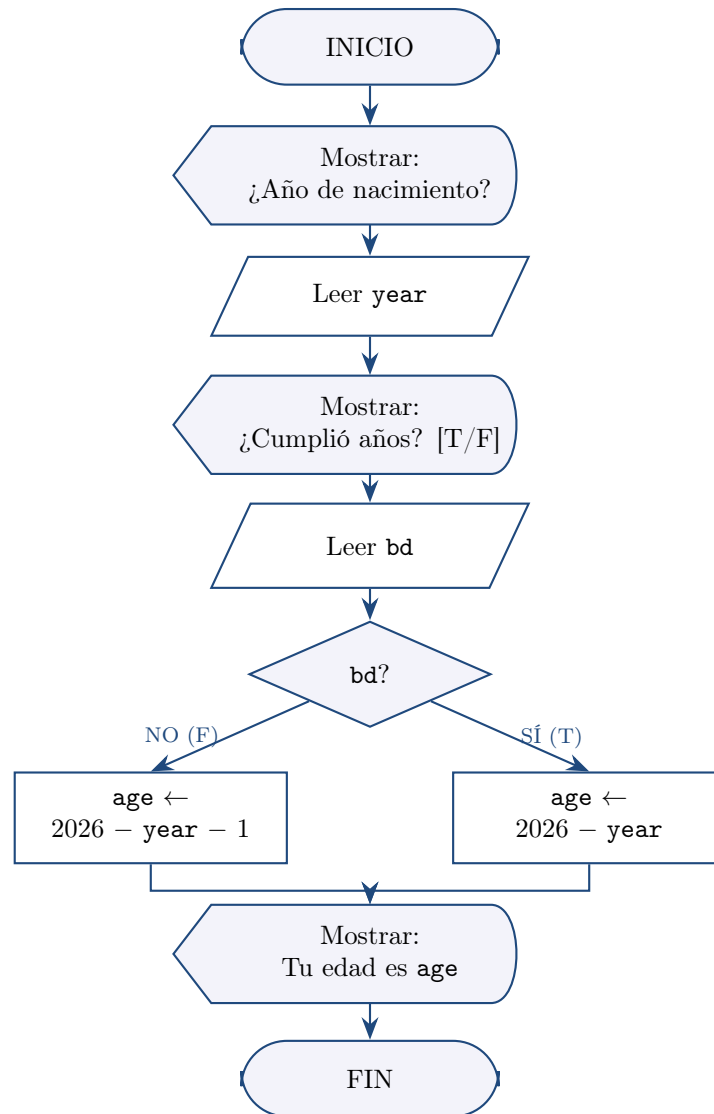
1 program operaciones
2   implicit none
3   integer :: year, age
4   logical :: bd           ! variable logica (booleana)
5
6   print *, "Digita tu año de nacimiento:"
7   read *, year
8   print *, "Ya cumpliste años en 2026?"

```

```
9      print *, "[T] - Si"
10     print *, "[F] - No"
11     read *, bd                ! se lee como T (verdadero) o F (falso)
12
13     age = 2026 - year - merge(0, 1, bd)
14     print *, "Tu edad es ", age, "."
15
16 end program operaciones
```

Note una diferencia importante frente a C++: en C++ un `bool` se comporta como un entero (`true` vale 1 y `false` vale 0), por lo que puede usarse directamente en una operación aritmética como `(1 - bd)`. En Fortran, en cambio, el tipo `logical` no es aritmético: sus únicos valores posibles son `.TRUE.` y `.FALSE.` (que se leen o escriben como T/F), y no puede sumarse ni restarse directamente. Por ello se usa la función intrínseca `merge(valor_si_verdadero, valor_si_falso, condición_lógica)`, que devuelve uno de dos valores numéricos según el contenido de la variable lógica.

El diagrama de flujo correspondiente es el siguiente (obsérvese cómo la función `merge` equivale, en el diagrama, a una decisión con dos caminos que luego convergen):



#### Ejercicio 4

En la página OnlineGDB, puede copiar el código anterior y ejecutarlo empleando el botón *Run*.

### Tarea

- Desarrolle el *pseudocódigo* y el diagrama de flujo para las siguientes tareas:
  - Sacar el libro “Cien años de soledad” de la Biblioteca Central.
  - La receta para una tortilla.
  - Almorzar en Cafetería Central.
- Diseñe un programa que calcule la altura máxima de un proyectil conociendo el ángulo con respecto a la horizontal y la rapidez inicial, empleando un diagrama de flujo y realice el código en PYTHON.

3. Haga un glosario con las palabras resaltadas en *itálicas*.